DEVELOPER REFERENCE

Git & GitHub Commands

What each command does, and how to use it, with real examples.

```
$ git commit -m "first step"
```

Bilal Nadeem

CONTENTS

What we will cover

01 Setup & starting a repo

02 Stage, commit, push & pull

03 Branches & remotes

04 History & undoing mistakes

05 Stash, rebase & tags

06 GitHub CLI & full workflow

01 · SETUP

Configure Git once

`git config --global user.name "Karamat"`	Name stamped on every commit
`git config --global user.email "you@mail.com"`	Email that links commits to GitHub
`git config --global init.defaultBranch main`	New repos start on the “main” branch
`git config --list`	Show all your current settings
`git --version`	Check that Git is installed

“--global” applies to every repo on your machine. Leave it out to set a value for one repo only.

01 · SETUP

Start a repository

`git init`	Turn the current folder into a repo
`git clone <url>`	Copy a GitHub repo to your machine
`git clone <url> my-folder`	Clone into a folder name you choose
`git clone git@github.com:user/repo.git`	Clone over SSH, no password prompts

```
# Example
$ git clone https://github.com/user/shop.git
$ cd shop
```

02 · EVERYDAY WORKFLOW

Stage and commit

`git status`	See what changed and what is staged
`git add index.php`	Stage one file for the next commit
`git add .`	Stage every change in the folder
`git commit -m "Add login validation"`	Save a snapshot with a message
`git commit -am "Fix typo"`	Stage tracked files and commit at once

```
# Example
$ git add . && git commit -m "Add cart page"
```

02 · EVERYDAY WORKFLOW

Sync with GitHub

`git push`	Upload your commits to GitHub
`git pull`	Fetch and merge the latest changes
`git fetch origin`	Download changes without merging
`git push -u origin main`	First push: link the branch to remote

```
# Start of the day: get the latest, then work
$ git pull
$ git push
```

Work on branches

`git branch`	List local branches
`git branch -a`	Include remote branches too
`git switch -c feature/cart`	Create a branch and switch to it
`git switch main`	Move to another branch
`git merge feature/cart`	Merge it into the current branch
`git branch -d feature/cart`	Delete a branch once it is merged

Manage remotes

`git remote -v`	List remotes and their URLs
`git remote add origin <url>`	Connect a local repo to GitHub
`git remote set-url origin <url>`	Change the URL, e.g. HTTPS to SSH
`git push origin --delete feature/cart`	Delete a branch on GitHub

```
# Publish a local project to an empty GitHub repo
$ git remote add origin https://github.com/user/shop.git
$ git push -u origin main
```

Inspect history

`git log --oneline --graph --all`	Compact, visual commit history
`git log -p routes/web.php`	Every change made to one file
`git diff`	Changes not yet staged
`git diff --staged`	Changes ready to commit
`git show a1b2c3d`	Full details of a single commit
`git blame routes/web.php`	Who last changed each line

Undo mistakes

`git restore file.php`	Discard local edits to a file
`git restore --staged file.php`	Unstage, but keep the edits
`git commit --amend -m "New message"`	Fix the last commit (before pushing)
`git reset --soft HEAD~1`	Undo last commit, keep the changes
`git reset --hard HEAD~1`	**Careful:** undo commit and delete changes
`git revert a1b2c3d`	Safe undo via a new, opposite commit

Stash work for later

`git stash`	Shelve uncommitted changes
`git stash -u`	Include untracked files as well
`git stash list`	See everything you have stashed
`git stash pop`	Re-apply the latest stash and remove it
`git stash drop stash@{1}`	Delete one stash

```
$ git stash # urgent bug? park your work, fix it
$ git stash pop # then bring your work back
```

Rebase and cherry-pick

`git rebase main`	Replay your branch on top of main
`git rebase -i HEAD~3`	Squash or edit the last 3 commits
`git rebase --continue`	Resume after fixing conflicts
`git rebase --abort`	Cancel and return to where you were
`git cherry-pick a1b2c3d`	Copy one commit onto this branch

⚠ **Golden rule:** never rebase commits that others have already pulled.

Tag releases

`git tag`	List all tags
`git tag v1.0.0`	Quick tag on the current commit
`git tag -a v1.0.0 -m "First release"`	Annotated tag with a message
`git push origin v1.0.0`	Push one tag to GitHub
`git push --tags`	Push every tag at once

Tags pushed to GitHub can become Releases with notes and downloadable files.

06 · GITHUB CLI

Meet gh

Repos, pull requests and issues on GitHub, without leaving the terminal.

```
$ gh auth login
```

06 · GITHUB CLI

Sign in and manage repos

`gh auth login`	Sign in to your GitHub account
`gh repo create my-app --private`	Create a new repo on GitHub
`gh repo clone user/repo`	Clone using just owner/name
`gh repo view --web`	Open the repo in your browser

```
# Publish the current folder as a new private repo
$ gh repo create shop --private --source=. --push
```

Pull requests and issues

`gh pr create --fill`	Open a PR from your current branch
`gh pr list`	List open pull requests
`gh pr checkout 42`	Check out PR #42 locally to test it
`gh pr merge 42 --squash`	Squash-merge PR #42
`gh issue create --title "Checkout bug"`	Open a new issue
`gh issue list`	List open issues

06 · FULL WORKFLOW

A feature, start to finish

STEP 1

Get the latest

```
git switch main
git pull
```

STEP 2

Create a branch

```
git switch -c feature/pdf
```

STEP 3

Commit your work

```
git add .
git commit -m "Add PDF"
```

STEP 4

Push the branch

```
git push -u origin feature/pdf
```

STEP 5

Open a pull request

```
gh pr create --fill
```

STEP 6

Clean up after merge

```
git switch main && git pull
git branch -d feature/pdf
```

EXTRAS

Handy extras

`echo "vendor/" >> .gitignore`	Tell Git to ignore a folder
`git rm --cached .env`	Stop tracking a file already committed
`git clean -fd`	**Careful:** deletes untracked files
`git reflog`	Recover “lost” commits after a reset
`git help <command>`	Built-in docs for any command

Commit small. Pull often. Branch for everything.

Clear messages

Say what changed and why, in the present tense.

Check first

Run git status and git diff before every commit.

Undo safely

Prefer revert over reset on shared branches.